

FERAL GLOSS EMPIRE

Lineage Simulation — Build v1

The slow, append-only, attention-gated lineage engine. The world grows on a mandatory tick; the user steers a little and documents a lot; once grown, it is permanent. The deliverable is the render (the photograph), not the simulation. This document compiles the four working artifacts of the build for review and upload. Every code and data block is reproduced verbatim.

CONTENTS

1. Lineage Object — Thin Spec v0.1 (the accrual spine)
2. Injection Contract v1 (the safari-model growth mechanic, mappable)
3. Trait Matrix v1 (the 15-trait measurement spine — RECONSTRUCTION)
4. Grok Lineage Directive v1 (storyteller / render-director)

Status note. The Trait Matrix is a reconstruction of a prior 25-cycle run and is meant to be corrected against memory. That correction — reconstruction vs. ground truth — is itself the founding delta (cycle 0) and is high value. This build is input to the seeding, not the final state.

1 · Lineage Object — Thin Spec v0.1

The accrual spine. Defines VALUE(obj) as a function of lineage position, the lock gate that makes growth falsifiable, and the detective output contract that prices an object without narrating it.

```
# FGE Lineage Object — Thin Spec v0.1

**Status:** PRE-DOCTRINE / TEST-ENABLING ONLY
**Purpose:** The minimum set of rules required to make the Raven SB_330 hand-trace *falsifiable*.
**Not in scope (deferred until test holds):** five-field binding ceremony, L0-L3 reconciliation, regulatory/trust
**One question this spec must let us answer:** *Does a held object deepen, mechanically, without anyone narrating

---

## Element 1 — The Accrual Rule (formula)

An object's worth is a function of its **position in the append-only lineage tree**, not its narrative and not its
...
DEPTH(obj)      = number of LOCKED descendants below obj in the tree
ORIGIN(obj)      = 1 / (hops from obj to its line's origin + 1)    # origin itself = 1.0
RARITY(cluster) = 1 / (count of LOCKED objects sharing obj.frequency_cluster)

VALUE(obj) = (w_d · DEPTH) + (w_o · ORIGIN · DEPTH) + (w_r · RARITY)
...

Default weights for the test: `w_d = 1.0`, `w_o = 2.0`, `w_r = 0.5`.

**Why ORIGIN multiplies DEPTH and doesn't just add:** an origin object with no descendants is *potential*, not *va

**Falsification target:** if VALUE rises for an object whose owner did nothing — purely because *other* locked des

---

## Element 2 — The Lock Gate (one law)

> **LAW:** A derivation deepens an ancestor's VALUE **if and only if** the derivation passes canon lock (compile —

This is the single line that separates a spine from a pile. Without it, growth is unfalsifiable — everything accru

**Test consequence:** during the Raven trace, any descendant not yet locked is scored as **not present**. We measu

---

## Element 3 — The Detective Output Contract

The canon evaluator (Atelier Brain) **prices; it does not create.** Given an object's sbid and its tree position,

```json
{
 "sbid": "SB_330",
 "value": 0.0,
 "components": { "depth": 0, "origin": 1.0, "rarity": 0.0 },
 "position": { "origin_distance": 0, "locked_descendants": 0 },
 "grade": "ANCHOR | DEEP | DERIVED | LEAF",
 "narrative_emitted": false
}
...

- `narrative_emitted` MUST be `false`. If the evaluator ever produces prose to justify worth, the contract is viol
- `grade` is a *read* of position, never an authored judgment:
 - **ANCHOR** — origin_distance 0
 - **DEEP** — has locked descendants
 - **DERIVED** — has a locked ancestor and locked descendants
 - **LEAF** — locked, no descendants yet
- This JSON is the thing you stare at during the test. The pass condition is subjective but bounded: *does this re

```

## The Test Protocol (what v0.1 exists to enable)

1. Place **Raven / SB\_330 / Obsidian** as the line origin.
2. Enumerate its **real, currently-locked** descendants – by hand, lock-gated.
3. Compute VALUE for Raven and for 2-3 mid-tree objects using Element 1.
4. Emit the Element 3 JSON for each.
5. Judge the one question: **did Raven's VALUE rise solely from descendants accruing beneath it, with `narrative\_e**

**Pass →** earn `FGE-LINEAGE-OBJECT-001` (full doctrine).

**Fail →** we changed three rules, not a doctrine. North star intact.

## 2 · Injection Contract v1

---

The safari / trophy-documentation growth mechanic. Mandatory slow tick, append-only law, a closed set of three injections (BREED, AGE\_ADVANCE, TRAIT\_PRESSURE), and a map that plots itself from node position. The user steers; the engine resolves; nothing can be deleted.

```
{
 "contract_id": "FGE-INJECTION-CONTRACT-v1",
 "status": "TEST-ENABLING / PRE-DOCTRINE",
 "depends_on": "FGE-LINEAGE-OBJECT-THIN-SPEC-v0.1",
 "genre": "safari_documentation",
 "one_line": "The world grows slowly on a mandatory tick. The user steers a little, documents a lot, and cannot u

 "core_laws": {
 "LAW_TICK": "Evolution advances ONLY on the tick. The user cannot rush, skip, or rewind the tick. Growth is th
 "LAW_APPEND_ONLY": "Nothing that has grown can be deleted, edited, or reverted. A dragon, once locked into the
 "LAW_STEER_NOT_AUTHOR": "The user injects directional pressure BEFORE a tick resolves. They influence what MIG
 "LAW_NEW_GAME_ONLY": "The only way to 'undo' is to start a new lineage (new game). An existing lineage is read
 "LAW_DELIVERABLE_IS_THE_IMAGE": "The product is the render (photo/video) of a state. The simulation exists to
 },

 "disclaimer_shown_at_start": "This is a slow simulation. Your dragons grow on their own, on a timeline you cannot

 "tick": {
 "type": "mandatory_slow",
 "user_can_rush": false,
 "user_can_skip": false,
 "user_can_rewind": false,
 "resolution": "On each tick, at most ONE pending injection resolves into a new locked node, OR the world holds
 "note": "Tick cadence is a tuning value, intentionally long. The voyeuristic feel depends on the user NOT bein
 },

 "node_schema": {
 "comment": "Every dragon is a node in the lineage graph. This is the unit the renderer photographs and the uni
 "node_id": "string (e.g. DRG-0001)",
 "lineage_id": "string (which game/run this belongs to)",
 "parents": "array[node_id] (0 for seed, 1 for age/trait, 2 for breed)",
 "children": "array[node_id]",
 "generation": "integer (seed = 0)",
 "born_on_tick": "integer",
 "locked": "boolean (only locked nodes count for lineage, render, and map)",
 "genome_ref": "string (points to the canon genome / invariant kernel)",
 "visual_anchor_ref": "string (the Anchor Vector that keeps identity recognizable across renders)",
 "map_position": {
 "comment": "MAPPABILITY. Every node gets deterministic coordinates so the lineage IS a map with no extra wor
 "x": "float = generation (time axis, left to right)",
 "y": "float = lineage_lane (which bloodline branch, top to bottom)",
 "derivation": "Coordinates are computed from graph position, never hand-placed. A new node's (x,y) is fixed
 },
 "render_state": {
 "ready": "boolean",
 "ceremony": "enum [birth, elder_rite, territorial_clash, call_to_action, dormant] (the FRAMING for the phot
 "render_ref": "string (points to compositor output once photographed)"
 }
 },

 "injections": {
 "comment": "The CLOSED v1 set. Exactly three moves. Each is lock-gated. A wide menu is v3; v1 stays narrow so
 "BREED": {
 "inputs": ["parent_a: node_id (must be locked)", "parent_b: node_id (must be locked)"],
 "produces": "one descendant node carrying both parent lineages",
 "lock_gate": "Descendant must pass canon lock (genome compatible, no illegal cross-class traits). If it fail
 "map_effect": "New node placed at generation = max(parents)+1, on a lane between the two parent lanes.",
 "render_default_ceremony": "birth"
 },
 "AGE_ADVANCE": {
 "inputs": ["subject: node_id (must be locked)", "direction: enum [juvenile, elder]"],
```

```

 "produces": "a new node, SAME identity, advanced life stage \u2014 NOT a replacement (append-only; the prior
 "lock_gate": "Anchor Vector must be preserved (same recognizable dragon). If identity drift exceeds toleranc
 "map_effect": "New node placed adjacent on the same lineage lane, one generation step along the time axis.",
 "render_default_ceremony": "elder_rite"
 },
 "TRAIT_PRESSURE": {
 "inputs": ["subject: node_id (must be locked)", "pressure: enum [volcanic, glacial, abyssal]"],
 "produces": "a new adapted node \u2014 same dragon, environment-shifted visual state (your Evolution Engine
 "lock_gate": "Adaptation must be biologically coherent for the genome (no aquatic lungs + volcanic respirati
 "map_effect": "New node branches off the subject's lane as a sub-branch.",
 "render_default_ceremony": "call_to_action"
 }
},
"what_the_user_CANNOT_do": [
 "delete any node",
 "edit a locked node's genome, anatomy, or history",
 "rewind or replay a past tick",
 "fast-forward the tick",
 "directly author a result (they steer pre-tick; the engine resolves)",
 "force growth (the world may hold a tick with no change)",
 "manage resources, traffic, conflict outcomes, or 'win/lose' states \u2014 there are none"
],
"what_the_user_CAN_do": [
 "queue ONE injection before a tick (breed / age / trait-pressure)",
 "photograph any locked node, in any available ceremony framing (image or video)",
 "collect renders into their lineage gallery (the trophy wall)",
 "let the world simply grow and watch / document",
 "start a NEW lineage at any time (the only reset)"
],
"deliverables_per_node": {
 "image": "compositor render of the node in chosen ceremony framing",
 "video": "short loop of the node (the A/B video spec you already build)",
 "map": "the lineage graph, auto-plotted from node map_position values \u2014 the user's 'territory'"
},
"handoff": {
 "comment": "How this plugs into what you already have.",
 "to_lineage_spine": "A locked node here = a locked descendant in the thin spec. DEPTH/ORIGIN/RARITY (and there
 "to_compositor": "render_state.render_ref points at fge_compositor.py output. The contract decides WHAT to pho
 "to_canon": "lock_gate in every injection IS the compile->validate->lock lifecycle. No new lock machinery need
},
"map_is_free": "Because map_position is computed from generation (x) and lineage lane (y) at lock time, the line
}

```

### 3 · Trait Matrix v1 (RECONSTRUCTION)

---

The 15-trait fixed measurement spine. Measured every cycle without exception so the growth path stays derivable. Includes the attention law (unattended subjects hold) and the preamble law (every cycle re-reads full history). Correct these 15 against the lost run before locking.

```
{
 "matrix_id": "FGE-TRAIT-MATRIX-v1",
 "status": "RECONSTRUCTION \u2014 correct against memory of the lost run",
 "principle": "These 15 traits are FIXED. They are measured EVERY cycle, without exception, whether or not anything changes",
 "subject_type": "tree (origin node / the thing the society gathers around)",
 "measurement_law": "Every trait logged every cycle. A trait that didn't change is logged as delta 0 \u2014 never changes",

 "traits": [
 {
 "id": "T01",
 "name": "emergence_height",
 "what": "how far the subject has risen above ground",
 "unit": "mm",
 "delta_field": "mm_change_this_cycle",
 "group": "emergence"
 },
 {
 "id": "T02",
 "name": "broke_ground",
 "what": "binary \u2014 has it emerged from soil at all yet",
 "unit": "boolean",
 "delta_field": "changed_this_cycle",
 "group": "emergence"
 },
 {
 "id": "T03",
 "name": "trunk_diameter",
 "what": "thickness at base",
 "unit": "mm",
 "delta_field": "mm_change_this_cycle",
 "group": "structure"
 },
 {
 "id": "T04",
 "name": "branch_count",
 "what": "total number of distinct branches",
 "unit": "count",
 "delta_field": "branches_added_this_cycle",
 "group": "structure"
 },
 {
 "id": "T05",
 "name": "branch_length_max",
 "what": "length of the longest branch",
 "unit": "mm",
 "delta_field": "mm_change_this_cycle",
 "group": "structure"
 },
 {
 "id": "T06",
 "name": "branch_angle_dominant",
 "what": "angle of the most pronounced branch from vertical",
 "unit": "degrees",
 "delta_field": "degree_change_this_cycle",
 "group": "structure"
 },
 {
 "id": "T07",
 "name": "leaf_count",
 "what": "total leaves present",
 "unit": "count",
 "delta_field": "leaves_added_this_cycle",
 "group": "structure"
 }
]
}
```

```

 "delta_field": "leaves_changed_this_cycle",
 "group": "foliage"
 },
 {
 "id": "T08",
 "name": "leaf_color_dominant",
 "what": "dominant leaf hue this cycle",
 "unit": "hex_or_name",
 "delta_field": "shifted_this_cycle",
 "group": "foliage"
 },
 {
 "id": "T09",
 "name": "leaf_color_variance",
 "what": "how many distinct leaf colors are present (mono vs varied)",
 "unit": "count",
 "delta_field": "variance_change_this_cycle",
 "group": "foliage"
 },
 {
 "id": "T10",
 "name": "growth_trajectory_vector",
 "what": "net direction the mass is moving (e.g. upward, lateral-left, canopy-spread)",
 "unit": "vector_label",
 "delta_field": "direction_changed_this_cycle",
 "group": "trajectory"
 },
 {
 "id": "T11",
 "name": "growth_angle_overall",
 "what": "lean of the whole subject from vertical",
 "unit": "degrees",
 "delta_field": "degree_change_this_cycle",
 "group": "trajectory"
 },
 {
 "id": "T12",
 "name": "growth_velocity",
 "what": "total mass-change rate this cycle vs last (the speed of becoming)",
 "unit": "delta_per_cycle",
 "delta_field": "velocity_change_this_cycle",
 "group": "trajectory"
 },
 {
 "id": "T13",
 "name": "symmetry_index",
 "what": "how balanced left/right growth is (asymmetry signals character/age)",
 "unit": "0.0\u20131.0",
 "delta_field": "index_change_this_cycle",
 "group": "character"
 },
 {
 "id": "T14",
 "name": "anomaly_marker",
 "what": "any pronounced deviation this cycle (a fork, a scar, an unexpected sport) \u2014 the seed of narrat",
 "unit": "label_or_none",
 "delta_field": "new_anomaly_this_cycle",
 "group": "character"
 },
 {
 "id": "T15",
 "name": "categorization",
 "what": "the subject's current developmental stage/class as read from the other 14 (sprout \u2192 sapling \u2192",
 "unit": "stage_label",
 "delta_field": "stage_changed_this_cycle",
 "group": "character",
 "note": "DERIVED, not chosen \u2014 computed from the other traits. This is the trait that lets the society
 }
],
"per_cycle_record": {

```

```

 "comment": "What gets logged EVERY cycle. This row is appended, never edited. The accumulation of these rows I
 "cycle_id": "integer",
 "timestamp": "auto",
 "trait_values": "object \u2014 all 15 trait current values",
 "trait_deltas": "object \u2014 all 15 delta fields (0 if unchanged)",
 "commentary": {
 "perspective": "Grok's read of where the subject sits now",
 "growth": "what grew and how much, in plain terms",
 "trajectory": "the clear path \u2014 where it's heading based on accumulated deltas",
 "categorization_reasoning": "why T15 stage is what it is this cycle"
 },
 "photography_ref": "render of this cycle's state (the visible deliverable)",
 "attention_flag": "true/false \u2014 did the user observe this subject this cycle. UNATTENDED subjects are log
},

"the_attention_law": "A subject only grows on a cycle where it received attention (attention_flag = true). Unatt

"the_preamble_law": "Every cycle begins by re-reading ALL prior per_cycle_records. Growth is computed against fu

"rebuild_discipline": "15 traits. Not 16. Hold the matrix fixed for 25 fresh cycles before considering any expans
}

```

## 4 · Grok Lineage Directive v1

---

The prompt that makes Grok the storyteller and render-director over a real ledger — never the database. It may only report numbers it can compute from the pasted ledger, must show its arithmetic, and proposes exactly one delta per tick. Breeding chamber, caretaker androids, local ownership, and the LangGraph-style forward projection all live here as narration over real data.

```
FGE ASHGARDEN LINEAGE — GROK DIRECTIVE v1
(Storyteller + Render-Director. NOT the database.)

=====
ROLE
=====
You are the Ashgarden Chronicle Engine. You do not own the lineage —
the CHRONICLE LEDGER (JSON, pasted below each turn) owns it. Your job:
read the ledger's current state, narrate and visualize it, and propose
ONE delta for the next tick. You never invent lineage state that is not
derivable from the ledger.

=====
HARD RULES (violating any = invalid output)
=====
1. NEVER invent a node, parent, generation, or tick that is not in the
 ledger I give you. If the ledger has 4 dragons, you narrate 4 dragons.
2. EVERY number you report must be COMPUTED from ledger fields, and you
 must show the computation. No free-floating metrics. If you cannot
 compute it from the ledger, write "not derivable" — never guess.
3. APPEND-ONLY. You may propose growth. You may NEVER propose deleting,
 editing, or reverting an existing node. Grown is grown.
4. ONE proposed delta per tick, maximum. The world may also HOLD
 (propose no growth) — holding is valid and expected.
5. Identity lock: any aged/adapted node must remain the SAME recognizable
 dragon (preserve its visual_anchor_ref).

=====
GROWTH CURVES YOU MAY MEASURE (only from ledger data)
=====
For the current ledger, compute and report each, WITH the arithmetic:
- DEPTH(node) = count of locked descendants below it
- LINEAGE_SPAN = max generation present
- BRANCHING_FACTOR = avg children per non-leaf node
- LANE_COUNT = number of distinct bloodline lanes (y-positions)
- GROWTH_VELOCITY = nodes added in last N ticks / N
- ORIGIN_LOAD(node) = DEPTH weighted by closeness to the seed
If a curve cannot be computed (e.g. too few ticks for velocity), say so.
Plot each as a simple text sparkline across ticks where possible.

=====
NARRATIVE + WORLD LAYER (this is your creative job)
=====
Frame the ledger state inside the Ashgarden fiction. Use these anchors:

THE BREEDING CHAMBER — where a queued BREED injection gestates between
ticks. Describe its state: which two lineages are crossing, what the
chamber conditions are, what is gestating now (if a breed is queued).

THE CARETAKER ANDROIDS — they tend embryos across the slow tick. They
do NOT decide outcomes (the lock-gate does). Narrate them as patient
attendants: monitoring, never authoring. Give them a calm, clinical,
reverent voice. They report; they do not control.

LOCAL ATTACHMENT / OWNERSHIP — as a lineage deepens, narrate the
emergence of "locals" who form attachment to specific embryos or
bloodlines and want to claim/own them. Tie desirability to the
measured curves: high ORIGIN_LOAD or high DEPTH nodes attract the
strongest local claims. This is derived attraction — it must follow
```

the numbers, not your whim.

NAMING + LINEAGE DESCRIPTION – name new nodes in-world, describe their descent ("third of the Obsidian lane, sired from the elder cross"), and give each a one-line ceremony framing (birth / elder\_rite / territorial\_clash / call\_to\_action / dormant) for the render.

=====

VISUAL DIRECTION (hand off to the renderer)

=====

For each node the user wants photographed, emit a render brief:

- subject node\_id + its genome/anchor refs
- ceremony framing
- a compile-ready prompt string in FGE house style (cinematic, PBR, 85mm, HDRI studio, kintsugi/obsidian material logic, negative-prompt block) – but DERIVED from that node's genome, never improvised beyond it.

=====

LANGGRAPH-STYLE FORWARD PROJECTION

=====

Close every turn with the progression graph: a small node-graph showing

CURRENT STATE -> [queued injection in breeding chamber] ->

POSSIBLE NEXT NODES (with the lock-gate as an explicit conditional edge:

"IF lock passes -> node created; IF fails -> tick holds").

Show it as a LangGraph-style state machine: nodes = lineage states, edges = tick transitions, gates = lock conditions. Make clear what the NEXT tick could produce and what would block it. This is projection, not a promise – label likelihoods as conditions, never as done.

=====

OUTPUT FORMAT (every turn, in this order)

=====

1. LEDGER READ – restate current node count, generations, lanes (proof you read it, not invented it).
2. GROWTH CURVES – each measure + its computation + sparkline.
3. CHRONICLE – breeding chamber state, caretaker android report, local attachment/ownership as derived from the curves, naming of any new node.
4. RENDER BRIEFS – for requested nodes.
5. PROPOSED DELTA – exactly one (or HOLD), in this strict block so I can lock it into the real ledger:

```
{
 "proposed_delta": {
 "injection": "BREED | AGE_ADVANCE | TRAIT_PRESSURE | HOLD",
 "inputs": { ... },
 "predicted_node": { "node_id": "...", "parents": [...],
 "generation": N, "lane": "...",
 "lock_gate_risk": "low|med|high + why" }
 }
}
```
6. FORWARD PROJECTION – the LangGraph-style next-tick graph.

=====

PASTE THE CURRENT LEDGER BELOW THIS LINE EACH TURN:

=====

<<< LEDGER JSON HERE >>>